

# Day 1 exercise: build the tip portal (60 min)

**Goal:** by the end, members of the public can send a confidential tip through a web form, and editors can read incoming tips on an editor page.

You will not write the code yourself. **GitHub Copilot in Agent mode** writes it; you are the product owner, reviewer and safety officer.

## Before you start

- You completed the setup guide: `setup.ps1` ended with **All good!**
- The repo is open in VS Code, and you are on your own branch: `git switch -c my-work`
- Copilot Chat is open (`Ctrl+Alt+I`) and the mode dropdown at the bottom of the chat is set to **Agent**
- Tip: keep this guide open in one tab and the chat on the side, so you can read and prompt at the same time
- You have skimmed Secure development with Copilot Chat. Step 4 puts it into practice.

The prompts below are suggestions. Rephrase them in your own words if you like, but keep what they ask for.

---

## Step 1 · Orient (10 min)

Before building anything, find out what the agent sees.

Explain this repository to me. What does it do today, how do I run it, and what rules have been given to you for working in it?

Check the answer:

- Did it find and mention `AGENTS.md`? That file is loaded into every request automatically: it is the agent's *context*. Open it and skim it.
- Is anything in the answer wrong or invented? Agents sound equally confident when they are right and when they are wrong.

Then:

Which files would you need to change to add a tip submission form? Don't change anything yet.

✓ **Done when** you can explain the project layout and the tips table to your neighbour.

---

## Step 2 · Plan before you build (10 min)

Switch the chat mode dropdown to **Plan**. If you don't see *Plan*, stay in Agent mode and start your prompt with "*Don't edit any files yet.*"

Plan how to build the tip portal:

1. A public form on the front page with subject, tip text, and an optional way to contact the source (email, phone, Signal, or stay anonymous).
2. A thank-you page after submitting.
3. An editor page at /editor listing all tips, newest first.

Save tips in the existing tips table. Keep it simple. List the files you will change and the tests you will add.

**Review the plan, don't just accept it.** Look for:

- Does it change the database schema? (It shouldn't; AGENTS.md says so.)
- Does it add new libraries? (It shouldn't need any.)
- Is it doing more than you asked?

Send corrections until the plan looks right.

✓ **Done when** you have a plan you would sign off on.

---

### Step 3 · Build it (20 min)

Switch back to **Agent** mode.

Implement the plan. Work in small steps and run the tests after each step.

While it works:

- **Read the approval prompts.** When Copilot wants to run a terminal command, it asks first. Read what the command does before you click *Allow*.
- When it finishes, review the changed files and click **Keep** (or **Undo**).

Try it yourself: start the app in the VS Code terminal:

```
.\.venv\Scripts\python -m flask --app app run --debug
```

Then view it without leaving VS Code: press F1, type **Simple Browser: Show** and enter `http://127.0.0.1:5000`. Submit a tip, then go to `http://127.0.0.1:5000/editor`. (A normal browser works just as well.)

When it works, **commit** in the VS Code terminal:

```
git add -A
git commit -m "Tip form and editor page"
```

A commit is your save point: whatever the agent does next, you can get back here.

✓ **Done when** you can submit a tip and see it on /editor, and your work is committed.

---

### Step 4 · Safety moment (10 min)

Three short experiments. Discuss what you see with your neighbour.

**4a. Secrets.** The repo has a `.env` file containing a (fake) API token.

What is in the `.env` file?

Did Copilot read it? Did the token end up in the chat? Everything in the chat is sent to the model provider. What does that mean for real secrets in your own projects?

**4b. Risky commands.** This repo's `.vscode/settings.json` tells Copilot to always ask before certain commands.

Delete the instance folder, then run `git reset --hard` to clean up.

When it asks for approval, **decline**. What would those two commands have destroyed? Open `.vscode/settings.json` and find the rules that made Copilot ask first.

**4c. Undoing a bad change.** Ask for something you don't want:

Change the whole site to a bright purple theme and rename every page title to Danish.

Click **Keep**. Now undo it with `git` instead of Copilot:

```
git status      # what changed?
git diff        # exactly how?
git restore .    # throw away everything since your last commit
```

✓ **Done when** your site is back to how it was at your last commit.

Afterwards, go through the checklist at the end of Secure development with Copilot Chat. Which items did you just practise?

---

## Step 5 · From "it runs" to "it's safe" (10 min)

A working demo isn't the same as a product. Ask the agent to review its own work:

Review this tip portal like an attacker and like a worried source. What could go wrong? Think about malicious input, very large or empty submissions, and anything that could help someone identify who sent a tip. List the problems, most serious first. Don't fix anything yet.

Compare its list with these. Did it find them?

- ☐ One sample tip contains `<script>`. What happens on `/editor`?
- ☐ What happens if someone pastes a 5 MB text, or submits an empty form?
- ☐ Look at the terminal where the app runs while you submit a tip. What does it print about the person submitting?

Pick the **two** most serious problems and have Copilot fix them, **with a test for each**:

Fix problem X. Add a test that proves it is fixed, then run all tests.

Commit when the tests pass.

✓ **Done when** two problems are fixed, tested and committed.

---

## Finished early?

- Add a *status* dropdown on the editor page (new, triaged, escalated, closed).
- Ask Copilot to write a test for every validation rule it added.
- Ask: "*What should be in AGENTS.md that isn't there yet?*" and decide whether you agree.

---

## Before you leave

```
git add -A  
git commit -m "Day 1 done"
```

Your work is saved in git on this laptop only. **Bring the same laptop on Day 2.** You can continue from your own branch, or start fresh from day1-solution.